Programming Languages Notes

CG

April 30, 2026

1 Abstract syntax trees

An *abstract syntax tree* (AST) is a tree representation of the hierarchical structure of a program. Unlike a concrete parse tree, an AST omits purely syntactic detail such as parenthesis, semicolons used as separators, keywords whose meaning is already captured by the node type; retaining only the information needed to define the program's meaning.

Each *internal node* of the AST corresponds to a language construct (sequence, assignment, conditional, etc.) and each *leaf* corresponds to an atomic value or reference.

For an imperative language with mutable references, the relevant constructs are:

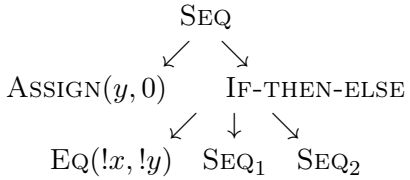
- $C_1 \,;\, C_2$, sequential composition
- $l := e$, assignment to location l
- **if** B **then** C_1 **else** C_2, conditional command
- **while** B **do** C, iteration
- $!l$, dereferencing a location (read the value stored at l)
- n, b, integer and boolean literals
- e_1 *op* e_2, arithmetic or boolean combination

1.1 solved example

Build the AST of the following program:

$y := 0 \,;$ **if** $!x = !y$ **then** $x := !x + 1 \,;\; y := !y - 1$ **else** $x := !x - 1 \,;\; y := !y + 1$

The top level operator is the semicolon that separates the two main components, so the root is a SEQ node. Its left child is the assignment $y := 0$, and its right child is the conditional. The conditional has three children: the guard $!x = !y$, the *then*-branch $x := !x + 1\,;\; y := !y - 1$, and the *else*-branch (symmetric).



note that $!x$ is represented as a DEREF node applied to the location x, which is distinct from x as an identifier: the location names a cell in the store, and dereferencing reads its content.

1.2 Exercises

- Build the AST of: $x := !x \times 2\;;$ **if** $!x > 10$ **then skip else** $x := !x + 1$
- Build the AST of: **while** $!i < !n$ **do** $s := !s + !i\,;\; i := !i + 1$
- Reconstruct the source program corresponding to the following AST: root is SEQ with left child ASSIGN$(x, 5)$ and right child IF-THEN-ELSE(EQ$(!x, 5)$, ASSIGN$(y, 1)$, Ass

2 Type inference

Type inference is the process by which a compiler deduces the type of an expression without explicit annotations from the programmer. the classical algorithm for the simply typed lambda calculus (and its extensions) is *Hindley Milner* (HM), which works by:

1. Assigning a fresh type variable α_i to each sub-expression whose type is unknown.
2. Generating *constraints* of the form $\tau_1 = \tau_2$ from the typing rules.
3. Solving the constraints by *unification*.

if unification fails, the program contains a type error; if it succeeds, the resulting substitution gives the most general type.

2.1 Typing rules for references

$$\text{T-Deref}\ \frac{\Gamma \vdash !l : T}{\Gamma \vdash l : \textbf{ref}\ T} \qquad \text{T-Assign}\ \frac{\Gamma \vdash l := e : \textbf{unit}}{\Gamma \vdash l : \textbf{ref}\ T \quad \Gamma \vdash e : T} \qquad \text{T-While}\ \frac{\Gamma \vdash \textbf{while}\ B\ \textbf{do}\ C : \textbf{unit}}{\Gamma \vdash B : \textbf{bool} \quad \Gamma \vdash C : \textbf{unit}}$$

2.2 solved example

$\lambda f.\, f\ f$ does not typecheck

Consider `proc(f)zero?((f f))`. We try to infer the type of f.

- Let $f : \alpha$.
- The expression $f\ f$ requires f to be a function type, so $\alpha = \alpha \to \beta$ for some fresh β.
- But $\alpha = \alpha \to \beta$ has no finite solution: expanding repeatedly yields $\alpha = (\alpha \to \beta) \to \beta = ((\alpha \to \beta) \to \beta) \to \beta = \cdots$

unification fails (occurs check). Therefore the expression has no type in the simply typed lambda calculus without recursive types.

2.3 Scala generics

```
case class MyPair[A,B](x:A,y:B)
val p = MyPair(1, "Scala")
def id[1](x:T)=x
val q = id(1)
```

the definition of `id` introduces a `polymorphic` type: it can be applied to any type , and each call site independently instantiates a T.

```
B  = !n < 5
B1 = !term <= 2
C1 = res := !n
C2 = c:=!a+!b; res:=!n; !c; a:=!b; b:=!c
C  = if B1 then C1 else C2; n:=!n+1
a:=1; b:=1; while B do C
```

reading off types from the operations performed:

- n, res, a, b, c : **ref int** (assigned integers and dereferenced in arithmetic).
- *term* : **ref int** (compared with 2).

- $B, B_1 : \mathbf{bool}$.
- $C_1, C_2, C : \mathbf{unit}$ (commands produce no value).
- The whole program : **unit**.

2.4 Exercises

Infer the type of the S combinator: $\lambda x.\, \lambda y.\, \lambda z.\; x\, z\, (y\, z)$. What constraints does unification generate?

Consider $\lambda f.\, \lambda x.\; f\, (f\, x)$. Does it typecheck? If so, give its most general type.

What types are inferred for `f`, `result`, and `pairs` in the following Scala snippet?

```
def f[A](lst: List[A]) = lst.head
val result = f(List(1,2,3))
val pairs  = List((1,"a"), (2,"b"))
```

Given $x : \mathbf{ref\ int}$ and $b : \mathbf{ref\ bool}$, does the following expression typecheck? If so, give its type.

$$b :=\!!x = 0 \;;\; \mathbf{if}\ !b\ \mathbf{then}\ x := 1\ \mathbf{else\ skip}$$

3 Operational semantics

3.1 Big-step semantics

In *big-step* (natural semantics, a judgement) $\langle e,\, \sigma\rangle \Downarrow v$ asserts that expression e in store σ evaluates *directly* to value v, without exposing intermediate steps. Derivations are trees whose leaves are axioms and whose internal nodes are inference rules.

3.1.1 Lists: `concat` and `append`

We assume the constructors **nil** (empty list) and $\mathbf{cons}(h, t)$ (list with head h and tail t).

`concat`(e, ℓ), insert element e at the end of ℓ.

Concat-Nil
$$\langle \mathbf{concat}(e, \mathbf{nil}),\, \sigma\rangle \Downarrow \mathbf{cons}(e, \mathbf{nil})$$

Concat-Cons
$$\frac{\langle \mathbf{concat}(e, \mathbf{cons}(h, t)),\, \sigma\rangle \Downarrow \mathbf{cons}(h, t')}{\langle \mathbf{concat}(e, t),\, \sigma\rangle \Downarrow t'}$$

append(ℓ_1, ℓ_2) concatenate two lists.

$$\textsc{Append-Nil}\quad \langle \mathbf{append}(\mathbf{nil}, \ell_2),\, \sigma\rangle \Downarrow \ell_2 \qquad \textsc{Append-Cons}\quad \frac{\langle \mathbf{append}(\mathbf{cons}(h, t_1), \ell_2),\, \sigma\rangle \Downarrow \mathbf{cons}(h, \ell')}{\langle \mathbf{append}(t_1, \ell_2),\, \sigma\rangle \Downarrow \ell'}$$

Derivation for append We verify $\langle \mathbf{append}(\mathbf{cons}(1, \mathbf{nil}), \mathbf{cons}(2, \mathbf{nil})),\, \sigma\rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))$:

$$\dfrac{\textsc{Append-Cons}\quad \langle \mathbf{append}(\mathbf{cons}(1, \mathbf{nil}), \mathbf{cons}(2, \mathbf{nil})),\, \sigma\rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))}{\textsc{Append-Nil}\quad \langle \mathbf{append}(\mathbf{nil}, \mathbf{cons}(2, \mathbf{nil})),\, \sigma\rangle \Downarrow \mathbf{cons}(2, \mathbf{nil})}$$

3.2 Small-step semantics

In *small-step* semantics, a judgement $\langle P, \sigma\rangle \to \langle P', \sigma'\rangle$ performs a single atomic reduction. A program is fully evaluated when no rule applies (it is a value or a stuck state) or when it reduces to **skip** (for commands).

3.2.1 For-loops in SIMP

The grammar of SIMP extended with **for** is:

$$C ::= l := E \mid C\,;\, C \mid \mathbf{if}\ B\ \mathbf{then}\ C\ \mathbf{else}\ C \mid \mathbf{while}\ B\ \mathbf{do}\ C \mid \mathbf{skip} \mid \mathbf{for}\ (C;\ B;\ C)\ \{C\}$$

The cleanest approach is to treat **for** as *syntactic sugar*:

$$\mathbf{for}\ (C_i;\ B;\ C_s)\ \{C_b\} \triangleq C_i\,;\, \mathbf{while}\ B\ \mathbf{do}\ C_b\,;\, C_s$$

If the language requires explicit small-step rules (no sugar), we give:

$$\textsc{For-Init}\quad \frac{\langle \mathbf{for}\ (C_i;\ B;\ C_s)\ \{C_b\},\, \sigma\rangle \to \langle \mathbf{for}\ (C_i';\ B;\ C_s)\ \{C_b\},\, \sigma'\rangle}{\langle C_i,\, \sigma\rangle \to \langle C_i',\, \sigma'\rangle}$$

$$\textsc{For-Skip}\quad \langle \mathbf{for}\ (\mathbf{skip};\ B;\ C_s)\ \{C_b\},\, \sigma\rangle \to \langle \mathbf{if}\ B\ \mathbf{then}\ (C_b\,;\, C_s\,;\, \mathbf{for}\ (\mathbf{skip};\ B;\ C_s)\ \{C_b\})\ \mathbf{else}\ \mathbf{skip},\, \sigma\rangle$$

Rule For-Init reduces the initialiser one step at a time. Rule For-Skip fires once the initialiser has been fully executed: it checks the guard and either runs the body followed by the step expression (and loops), or terminates with **skip**.

3.3 Exercises

- Write big-step rules for $\mathbf{reverse}(\ell)$, which reverses a list.
- Use `concat` as a helper.

  Prove with a full derivation tree:

  $$\langle \mathbf{concat}(3, \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))), \sigma \rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{cons}(3, \mathbf{nil})))$$

- Write big-step rules for $\mathbf{length}(\ell)$, which returns the number of elements, and evaluate $\langle \mathbf{length}(\mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))), \sigma \rangle \Downarrow 2$ with a derivation tree.

4 Lambda calculus

5 Type safety

6 Denotational semantics

7 Axiomatic semantics (Hoare logic)